UNIVERSITÉ DE ROUEN NORMANDIE

UFR des Sciences et Techniques

MASTER INFORMATIQUE - M1

Année Universitaire 2025-2026

Quoridor

Intelligence Artificielle et Interface Graphique

U.E. Théorie des Jeux

Projet de développement

Auteurs :
Mouhoubi Leiticia
Kherbachi Rima
Brahimi Thiziri
Cherifi Yasmine

Enseignant :
Selmi Carla

22 mars 2026

Table des matières

1 Introduction **2**

2 Technologies Utilisées **2**

3 Architecture du Projet **2**

4 Modèle de Données et Règles **2**
4.1 Structures de Données 2
4.1.1 Pion 3
4.1.2 Barriere 3
4.1.3 Plateau 3
4.2 Règles d'évolution 4

5 Intelligence Artificielle **4**
5.1 Optimisations de l'IA 4

6 Interface Graphique et Fonctionnalités **5**
6.1 Menu Principal 5
6.2 Règles du Jeu 6
6.3 Paramétrage et Lancement 7
6.4 Plateau de Jeu (In-Game) 8
6.5 Exemples de Parties (Replay) 9

7 Conclusion **10**

1 Introduction

Ce document présente le rapport détaillé du projet de développement du jeu **Quoridor**. Le projet consiste en une implémentation complète des règles officielles du jeu, couplée à une Intelligence Artificielle (IA) performante permettant de jouer contre l'ordinateur à différents niveaux de difficulté. Le tout est encapsulé dans une interface graphique développée en Java Swing.

2 Technologies Utilisées

Le projet repose sur un socle technique moderne et robuste :

- **Java 21** : Langage principal du projet, offrant des performances optimales et des fonctionnalités récentes.
- **Spring Boot 3.2** : Utilisé pour l'injection de dépendances (IoC - Inversion of Control), facilitant la gestion des différents services (Jeu, IA, Règles).
- **Swing** : API graphique intégrée au JDK pour la réalisation de l'interface utilisateur de bureau.
- **JUnit 5** : Framework de tests pour garantir la fiabilité de la logique métier (règles, algorithmes de parcours de graphe, IA).
- **Maven** : Outil de gestion de la construction et des dépendances.

3 Architecture du Projet

L'architecture de l'application est divisée en plusieurs couches fonctionnelles selon le modèle MVC (Modèle-Vue-Contrôleur) étendu :

- **Modèle (`model`)** : Représente les entités du jeu (`Plateau`, `Pion`, `Barriere`, `EtatJeu`, `Joueur`).
- **Services (`service`)** : Gère la logique métier, la validation des coups (`RegleService`), la gestion du déroulement de la partie (`GameService`) et le calcul des plus courts chemins (`PathService` à l'aide de l'algorithme BFS).
- **Intelligence Artificielle (`ai`)** : Implémente les algorithmes de prise de décision pour les joueurs informatisés (Minimax, Alpha-Bêta, etc.).
- **Interface Utilisateur (`ui`)** : Composants Swing gérant l'affichage graphique interactif et la captation des actions de l'utilisateur.

4 Modèle de Données et Règles

Le plateau est composé de 9×9 cases. Chaque joueur dispose d'un pion et de 10 barrières au début de la partie.

4.1 Structures de Données

Afin de modéliser l'état du jeu efficacement pour l'IA, plusieurs classes métiers ont été développées.

4.1.1 Pion

La classe `Pion` stocke la position actuelle sur le plateau.

```java
public class Pion {
    private int row;
    private int col;
    private Joueur joueur;

    public Pion(int row, int col) {
        this.row = row;
        this.col = col;
    }
    // Getters, Setters, hashCode et equals non-inclus pour la bri vet ...
}
```

4.1.2 Barriere

Représente une barrière placée sur le plateau. Le placement de la barrière repose sur ses coordonnées d'origine et son orientation.

```java
public class Barriere {
    public enum Orientation {
        HORIZONTAL, VERTICAL
    }

    private int row;
    private int col;
    private Orientation orientation;

    public Barriere(int row, int col, Orientation orientation) {
        this.row = row;
        this.col = col;
        this.orientation = orientation;
    }
}
```

4.1.3 Plateau

La structure centrale enregistre les listes de barrières et détermine si une case est atteignable.

```java
public class Plateau {
    private List<Barriere> barrieres;

    public Plateau() {
        this.barrieres = new ArrayList<>();
    }

    public boolean estDansPlateau(int row, int col) {
```

```
        return row >= 0 && row < Constantes.TAILLE_PLATEAU && col
            >= 0 && col < Constantes.TAILLE_PLATEAU;
    }

    public void ajouteBarriere(Barriere barriere) {
        this.barrieres.add(barriere);
    }
}
```

4.2 Règles d'évolution

À son tour, un joueur a deux options :

1. **Déplacer son pion** d'une case (horizontalement ou verticalement). Si les pions se font face, le joueur peut sauter par-dessus son adversaire (voire en diagonale si une barrière se trouve derrière lui).
2. **Placer une barrière** pour ralentir son adversaire. La barrière s'intercale exactement entre les tuiles.

La règle fondamentale du Quoridor (le concept de non-enfermement) a été implémentée *via* une recherche de parcours en largeur (BFS) : chaque placement de barrière a fait l'objet d'une validation stricte afin de garantir qu'aucun chemin vers la victoire n'est complètement obstrué pour l'un des joueurs.

5 Intelligence Artificielle

Le composant central de ce projet est l'Intelligence Artificielle. Plusieurs algorithmes d'exploration d'arbre de jeu ont été développés :

— **Alpha-Bêta** : Optimisation classique de l'algorithme Minimax, tronquant les branches de l'arbre qui ne peuvent pas influencer la décision finale, réduisant drastiquement le temps de calcul.

— **Négalpha-bêta (*Nég*$\alpha\beta$)** : Variante factorisée de l'Alpha-Bêta tirant parti du principe de jeu à somme nulle entre deux adversaires opposés.

— **SSS*** : Un algorithme basé sur l'A* et la résolution de graphes d'états, offrant de meilleures performances théoriques que l'Alpha-Bêta sur certains arbres spécifiques grâce à sa nature *best-first search*.

5.1 Optimisations de l'IA

Afin d'atteindre des profondeurs de recherche satisfaisantes (jusqu'à une profondeur de 5 coups d'anticipation ou plus pour le niveau Difficile), deux structures de données avancées ont été intégrées :

1. **Hachage de Zobrist (`ZobristHasher`)** : Méthode permettant de générer une signature/clé numérique unique pour chaque configuration du plateau de jeu de manière paramétrique et extrêmement rapide (opérations XOR uniquement).

```
@Component
public class ZobristHasher {
    private long[][][] pionZobrist;
```

```java
    private long[][][] barriereZobrist;
    private long tourZobrist;

    public long hash(EtatJeu etat) {
        long h = 0;

        for (int i=0; i<etat.getJoueurs().size(); i++) {
            Joueur j = etat.getJoueurs().get(i);
            h ^= pionZobrist[i][j.getPion().getRow()][j.getPion().getCol()];
        }

        for (Barriere b : etat.getPlateau().getBarrieres()) {
            int o = b.getOrientation() == Barriere.Orientation.HORIZONTAL ? 0 : 1;
            h ^= barriereZobrist[o][b.getRow()][b.getCol()];
        }

        if (etat.getIndexJoueurCourant() == 1) {
            h ^= tourZobrist;
        }

        return h;
    }
}
```

2. **Table de Transposition** : Cache mémorisant les évaluations et profondeurs relatives des états de jeu déjà explorés et évalués (indexés par les clés de Zobrist). Si l'algorithme aborde une même configuration par une autre série de coups, il récupère immédiatement le meilleur coup pré-calculé sans ré-explorer ces branches, accélérant logarihmiquement l'arbre.

6 Interface Graphique et Fonctionnalités

L'interface a été conçue en Java Desktop (Swing) et propose une ergonomie intuitive. Les différentes vues couvrent tous les aspects du cahier des charges.

6.1 Menu Principal

Le point d'entrée du jeu, permettant d'accéder aux différents sous-menus (Jouer, Règles, Exemples).

FIGURE 1 – Écran d'Accueil / Menu Principal

6.2 Règles du Jeu

Une vue dédiée à l'apprentissage interactif des règles officielles, rappelant les modes de déplacement, la gestion des sauts et le fonctionnement des barrières.

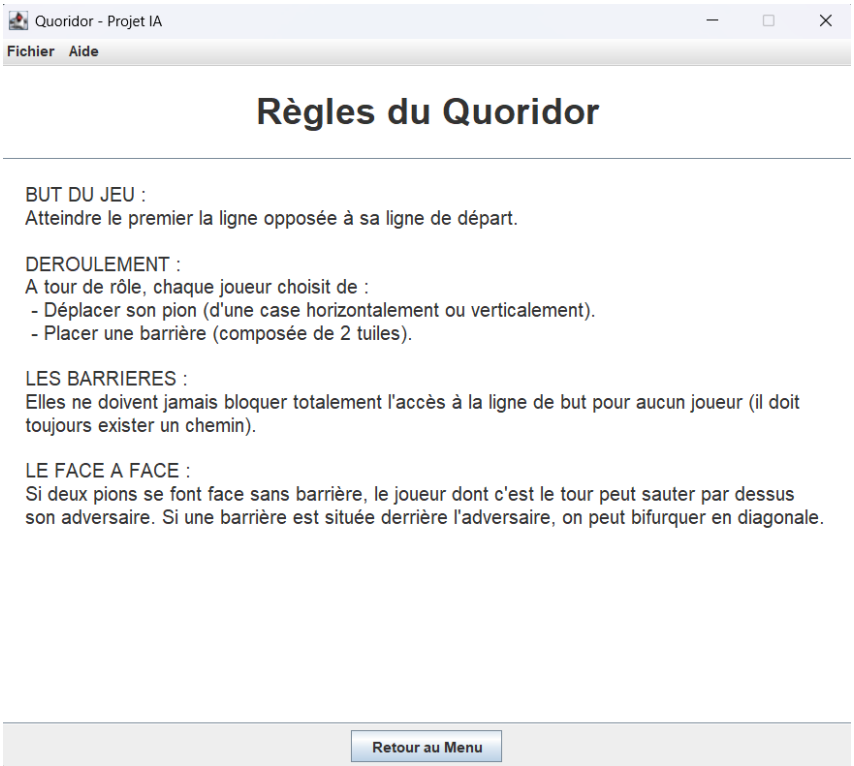

FIGURE 2 – Vue des Règles du Jeu intégrées

6.3 Paramétrage et Lancement

Ce panneau propose le choix du mode de jeu (Humain *vs* IA notamment), l'algorithme exact à exécuter et enfin la limitation de profondeur (niveau de difficulté).

FIGURE 3 – Configuration de la Partie et choix des heuristiques

6.4 Plateau de Jeu (In-Game)

L'interface de la partie en cours affiche le plateau interactif avec la position actuelle des pions (Bleu *vs* Rouge), des barrières noires, et un panel infromatif situé à droite rappelant le tour du joueur ou permettant d'annuler son dernier coup (*Undo*). Les emplacements atteignables sont affichés en surbrillance verte.

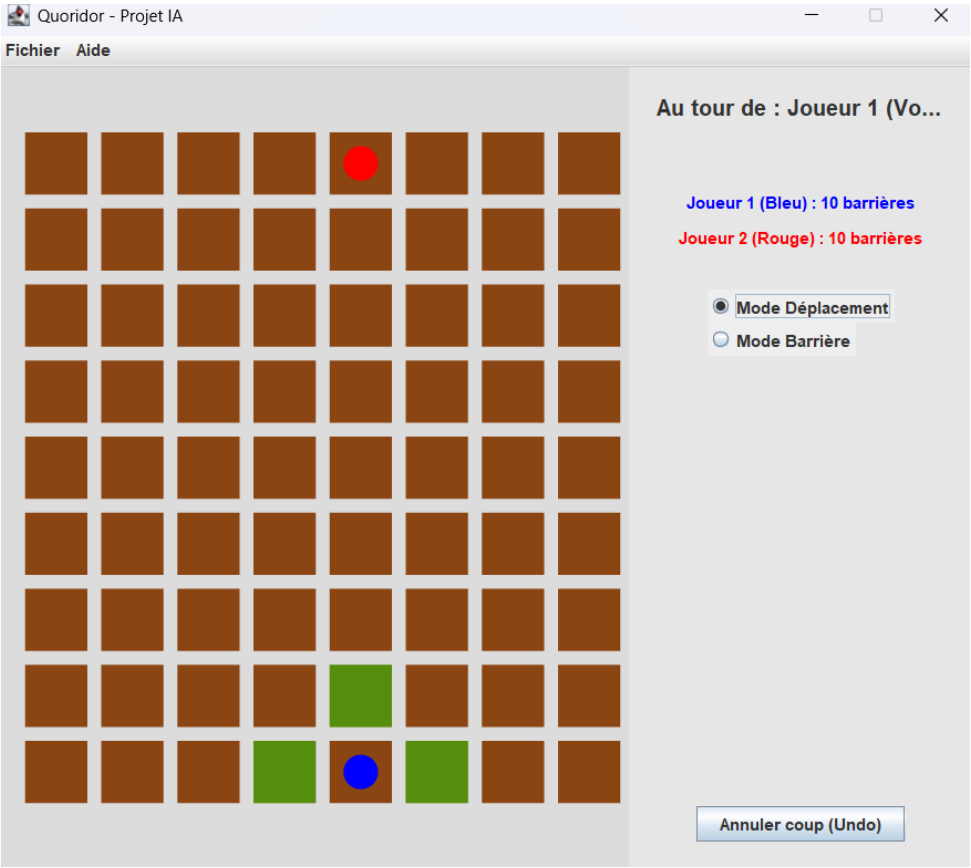

FIGURE 4 – Interface durant une partie en pleine action

6.5 Exemples de Parties (Replay)

Un module très puissant permettant de revisionner pas-à-pas une partie entière à blanc, avec des boutons de navigation (Précédent / Suivant), pour tester de manière statique la réponse du modèle de données et servir de didacticiel avancé.

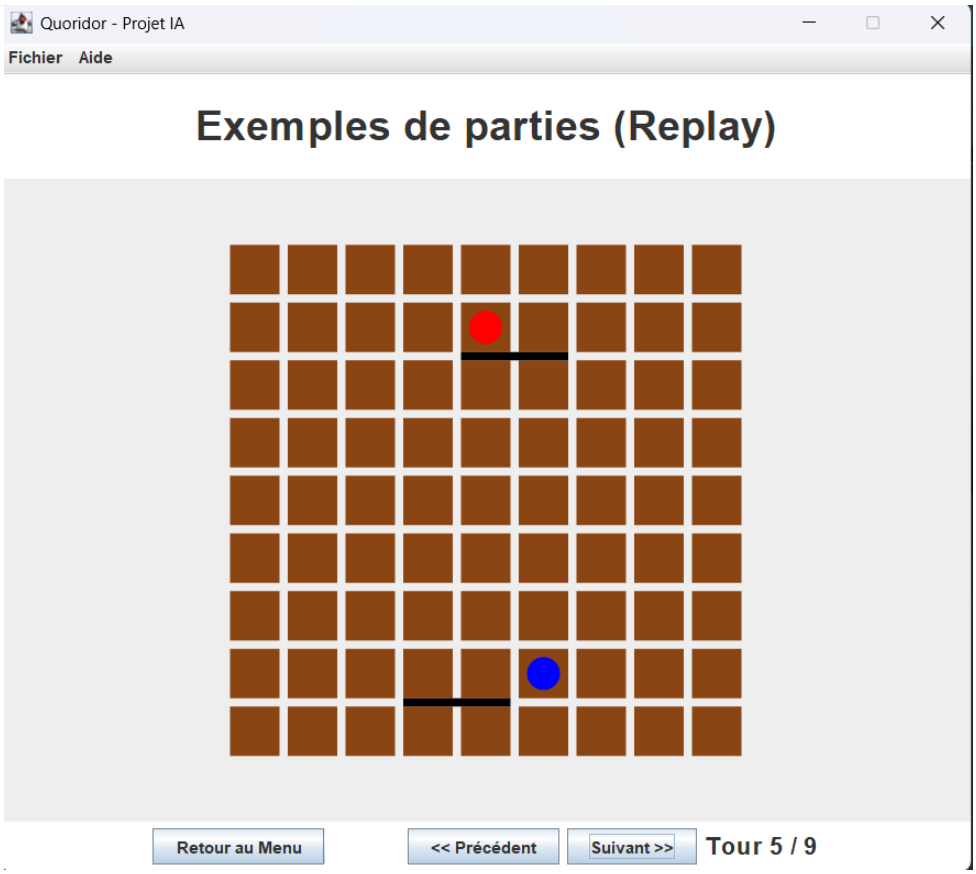

FIGURE 5 – Exemple de partie en mode "Replay"

7 Conclusion

Le développement de ce jeu complet certifie la mise en pratique d'algorithmes poussés de *Game AI*. La gestion pointilleuse des impératifs mathématiques (BFS de vérification, génération de hashs Zobrist) associée à l'arbre des possibilités (Minimax/Alpha-Bêta combiné aux tables de transposition) a procuré un défi analytique de grande envergure. L'usage de Spring Boot au sein d'une Stack Java monolithique a grandement facilité la liaison lâche entre les modèles de données IA et leurs rendus visuels (Swing), ouvrant amplement le code à d'éventuels ajustements ou au développement futur d'un *backend multiplayer*.